Programação OO

Leonardo Vitazik Neto

Católica de Santa Catarina
Centro Universitário

Agenda

- Apresentação do PEA;
- Introdução da disciplina;
- Configurar ambiente;
- Tutorial da IDE;
- Primeiro programa;
- Chamada e apresentação.

Apresentação do PEA

- Unidades;
- Avaliação;

Unidades

i. Introdução e fundamentos da Programação OO – 2 encontro;
ii. Elementos da Linguagem Java – 4 encontros;
iii. Orientação a Objetos – 6 encontros;
iv. Bibliotecas, projeto de soluções usando programação orientada a objetos – 7 encontros.

Metodologia de Ensino

- Aulas expositivas utilizando os recursos didáticos;
- Aulas práticas ou de exercícios;
- Trabalhos individuais ou em grupo.

AULAS TEÓRICAS

Aulas ministradas em sala de aula ou teleconferência nas quais a ênfase está em explicações conceituais.

AULAS PRÁTICAS

Aulas centradas na realização de atividades práticas pelos alunos com supervisão, orientação e auxílio do professor;
Aulas em que o professor realiza a resolução tutorada de exercícios (o professor conduz a resolução que é acompanhada pelos alunos); a
Alas em que o professor exemplifica a resolução de exercícios.
As aulas práticas incluem aulas de laboratório que são realizadas em ambientes específicos em que há uso de equipamentos e materiais que permitem a experimentação.

Avaliações

- 3 Notas:
  - 2 Provas;
  - Trabalho.
- Prova substitutiva.

Ambiente

- Instalação JDK;
- Instalação eclipse;

Tutorial da IDE

INSTALAÇÃO DO JDK EM AMBIENTE WINDOWS

Para instalar o JDK no Windows, primeiro baixe-o no site da Oracle. É um simples arquivo executável que contém o Wizard de instalação:

http://www.oracle.com/technetwork/java/

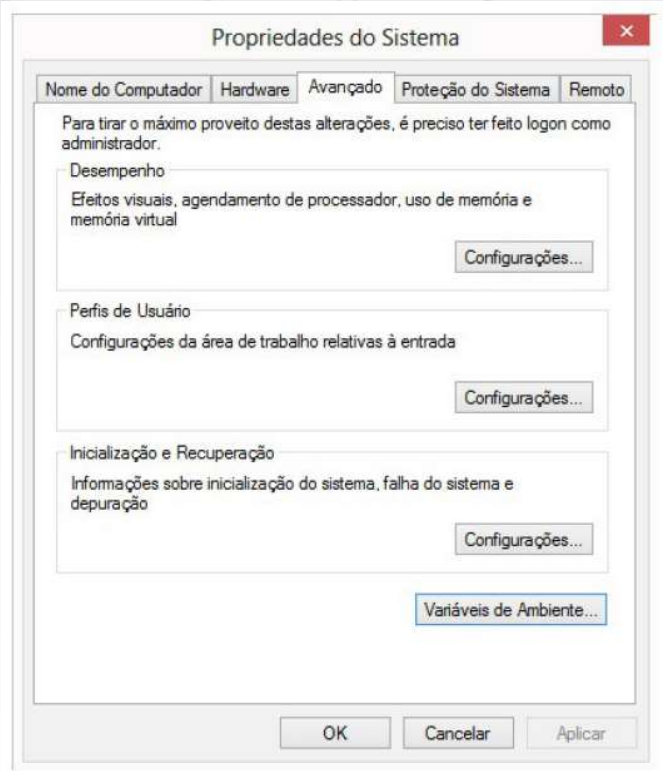
https://jdk.java.net/17/

https://adoptopenjdk.net/releases.html

Configurando o ambiente

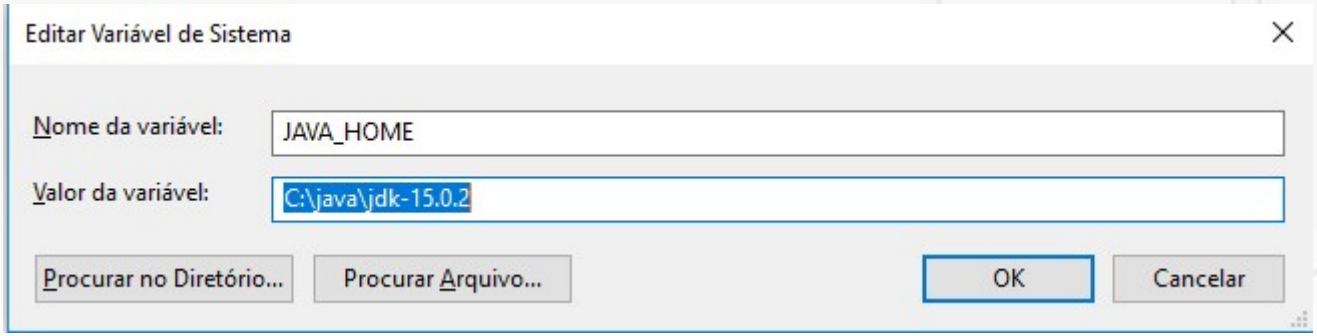
Precisamos configurar algumas variáveis de ambiente após a instalação, para que o compilador seja acessível via linha de comando. Caso você vá utilizar diretamente o Eclipse, provavelmente não será necessário realizar esses passos.

- Clique com o botão direito em cima do ícone *Computador* e selecione a opção *Propriedades*.
- Escolha a aba "Configurações Avançadas de Sistema" e depois clique no botão "Variáveis de Ambiente"



Configurando o ambiente

- Nesta tela, você verá, na parte de cima, as variáveis de ambiente do usuário corrente e, embaixo, as variáveis de ambiente do computador (servem para todos os usuários).
- Clique no botão Novo... Da parte de baixo.
- Em Nome da Variável digite JAVA_HOME e, em valor da variável, digite o caminho que você utilizou na instalação do Java. Provavelmente será algo como: C:\java\jdk-15.0.2



Configurando o ambiente

Não vamos criar outra variável, mas sim *alterar*. Para isso, procure a variável PATH, ou Path (dá no mesmo), e clique no botão de baixo "Editar".

Configurando o ambiente

Não altere o nome da variável! Deixe como está e adicione no final do valor ;%JAVA_HOME%\bin, não esqueça do ponto-e-vírgula - assim, você está adicionando mais um caminho à sua variável Path.

Configurando o ambiente

- Abra o prompt, indo em Iniciar, Executar e digite cmd .
- No console, digite javac -version . O comando deve mostrar a versão do Java Compiler e algumas opções.

```
C:\Users\leonardovn>javac -version
javac 15.0.2
```

O ECLIPSE

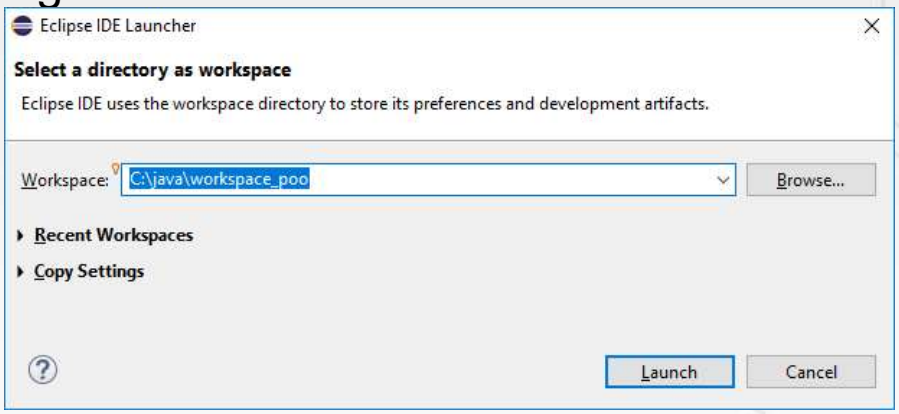
O Eclipse (http://www.eclipse.org) é uma IDE (integrated development environment). Diferente de uma RAD, onde o objetivo é desenvolver o mais rápido possível através do arrastar-e-soltar do mouse, onde montanhas de código são gerados em background, uma IDE te auxilia no desenvolvimento, evitando se intrometer e fazer muita mágica.

Veremos aqui os principais recursos do Eclipse. Você perceberá que ele evita ao máximo te atrapalhar e apenas gera trechos de códigos óbvios, sempre ao seu comando. Existem também centenas de plugins gratuitos para gerar diagramas UML, suporte a servidores de aplicação, visualizadores de banco de dados e muitos outros.

https://www.eclipse.org/downloads/packages/release/2020-12/r/eclipse-ide-java-developers

ECLIPSE

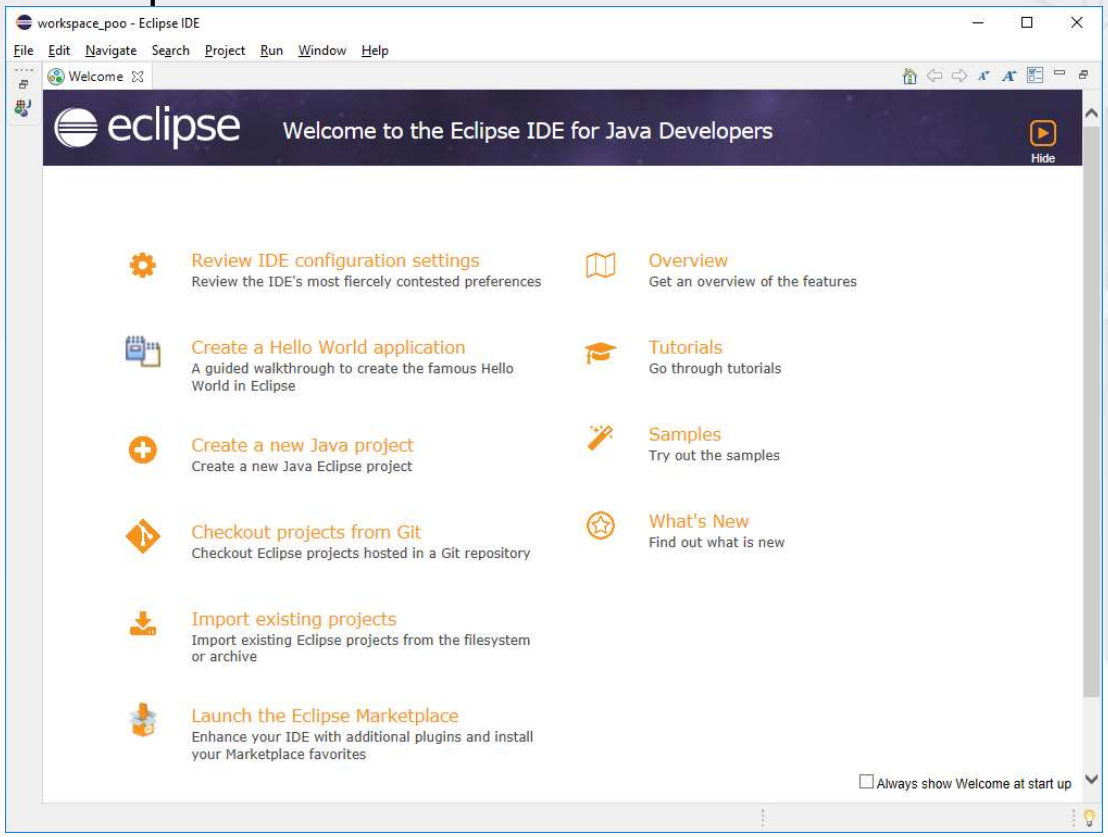
A primeira pergunta que ele te faz é que workspace você vai usar. Workspace define o diretório em que as suas configurações pessoais e seus projetos serão gravados.



Você pode deixar o diretório pré-definido.

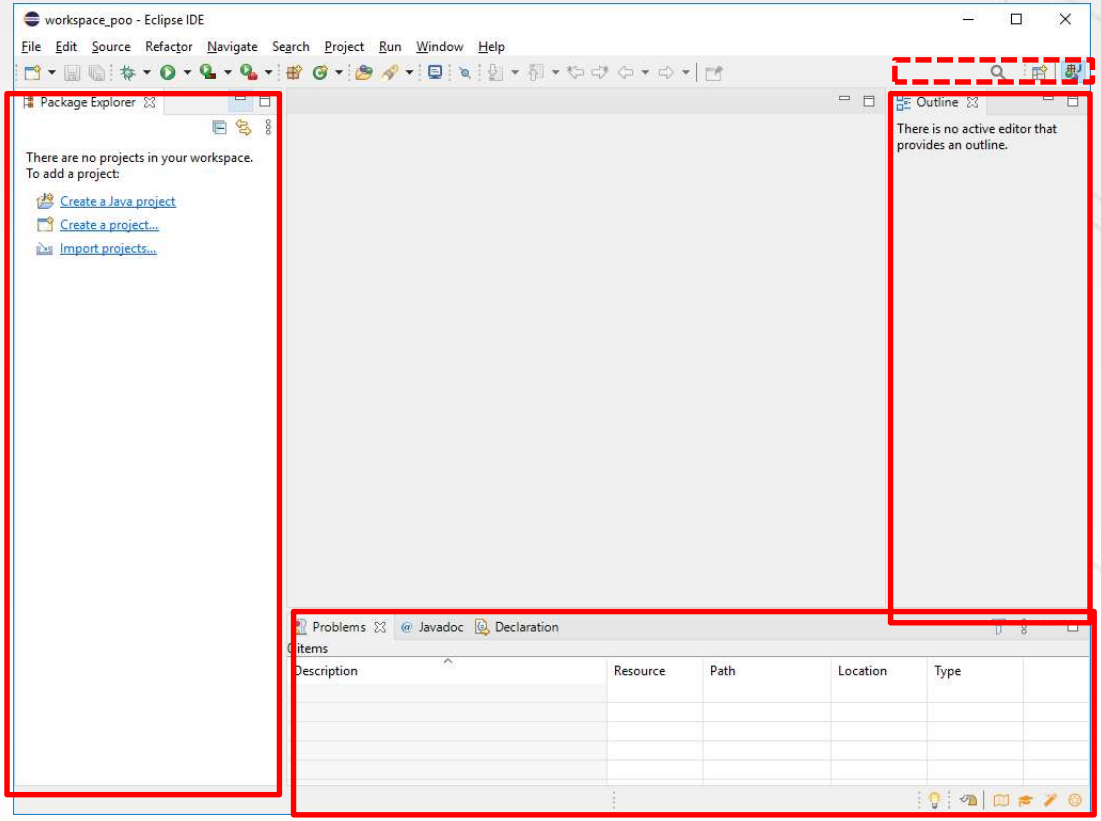
ECLIPSE

Logo em seguida, uma tela de Welcome será aberta, onde você tem diversos links para tutoriais e ajuda. Clique em Workbench.



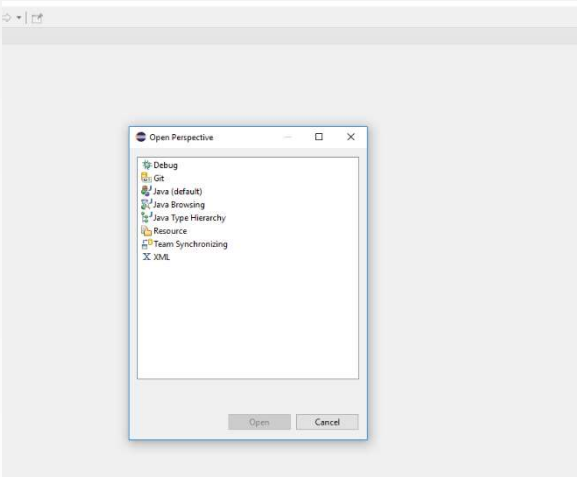
VIEWS E PERSPECTIVE

Feche a tela de Welcome e você verá a tela abaixo. Nesta tela, destacamos as Views (em linha contínua) e as Perspectives (em linha pontilhada) do Eclipse.



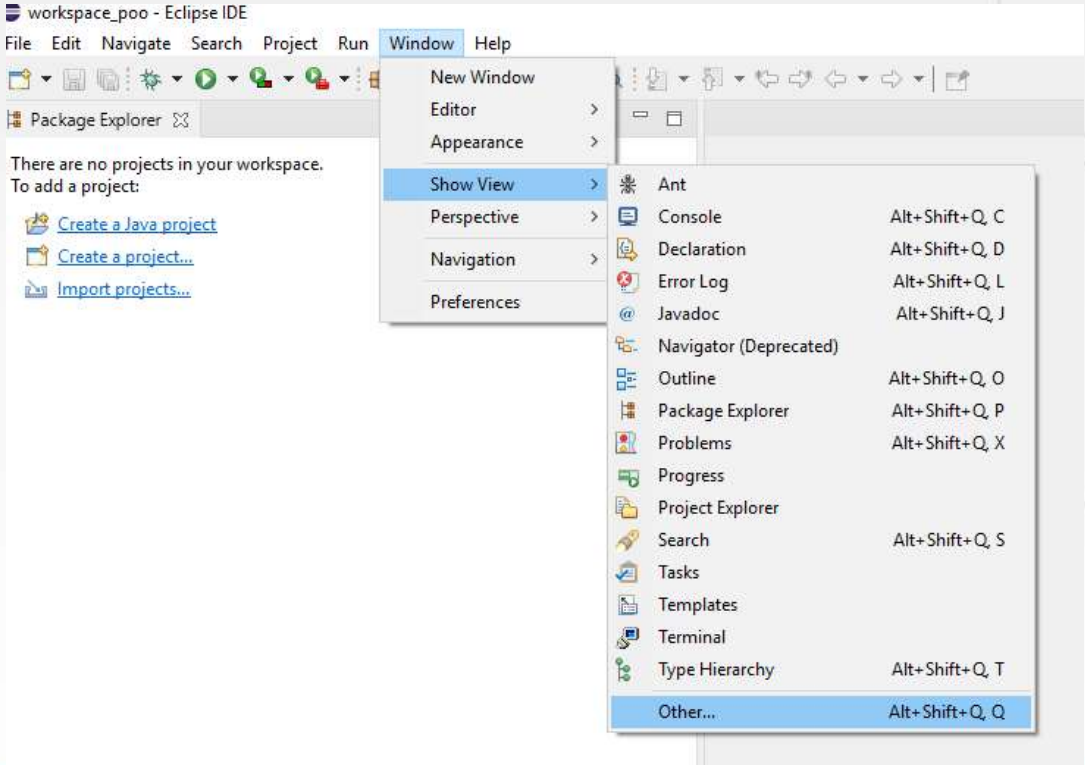
VIEWS E PERSPECTIVE

Mude para a perspectiva Resource, clicando no ícone ao lado da perspectiva Java, selecionando Open perspective. Neste momento, trabalharemos com esta perspectiva, antes da de Java, pois ela possui um conjunto de Views mais simples.



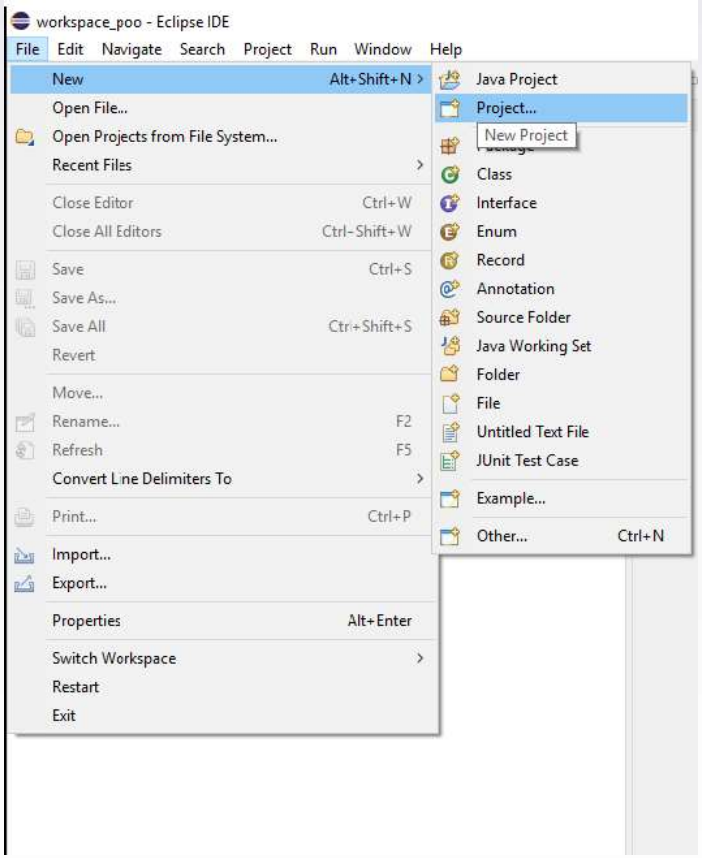
VIEWS E PERSPECTIVE

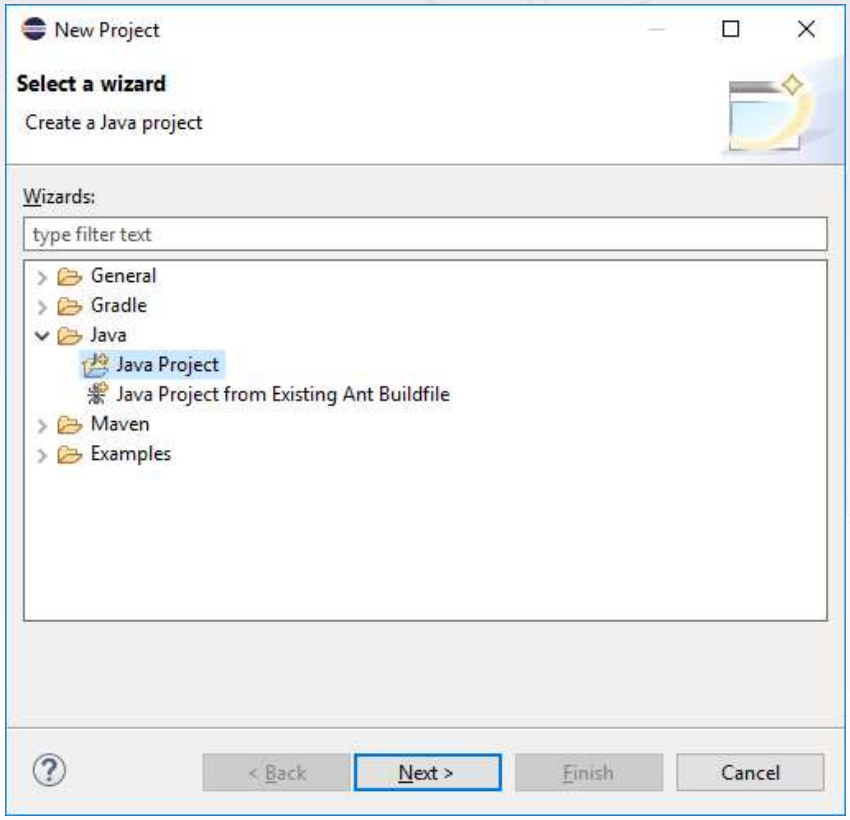
No menu **Window -> Show View -> Other**, você pode ver as dezenas de Views que já vem embutidas no Eclipse. Acostume-se a sempre procurar novas Views, elas podem te ajudar em diversas tarefas.



CRIANDO UM PROJETO NOVO

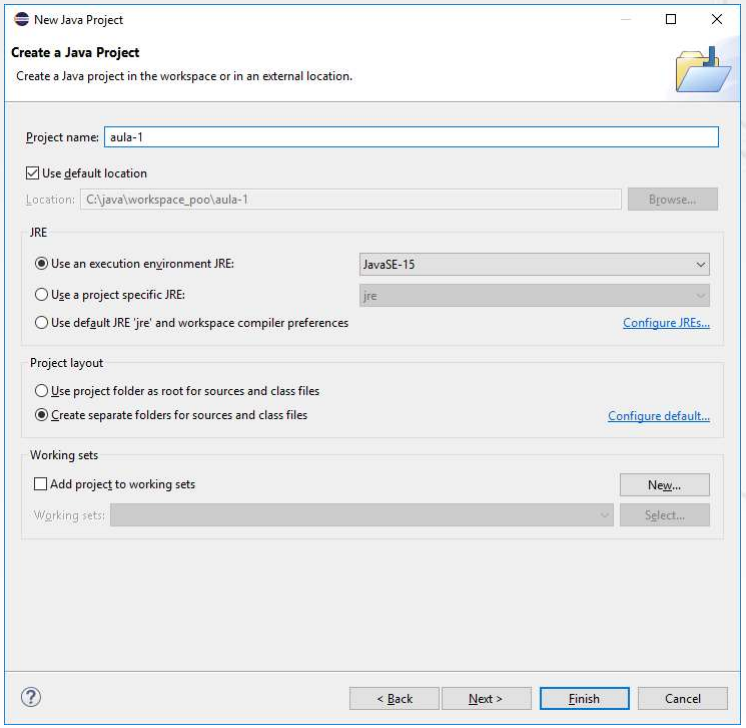
File -> New -> Project. Seleciona Java Project e clique em Next.





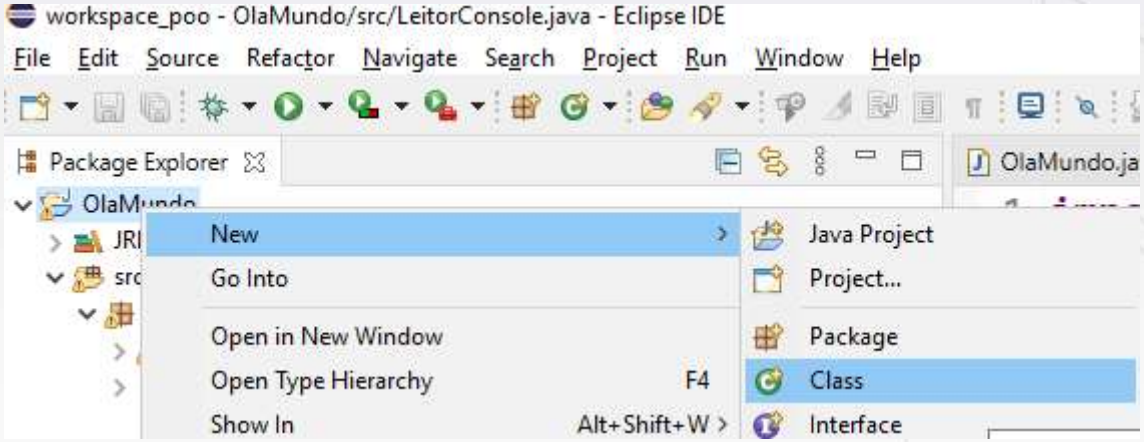
Projeto aula 1

- Crie um projeto chamado **aula-1**



Projeto aula 1

Crie uma nova classe chamada OlaMundo.



Code Assist

Vamos colocar um método main para testar nossa
Conta. Em vez de digitar todo o método main , vamos usar o **code assist** do Eclipse.
Escreva só main e aperte Ctrl + Espaço logo em seguida.

*OlaMundo.java LeitorConsole.java

```
1
2 public class OlaMundo {
3
4     main
5 }
6
```

main - main method
Main - com.sun.tools.javac.launcher
Main - com.sun.tools.javac
Main - com.sun.tools.javac.main
Main - com.sun.tools.javap
Main - com.sun.tools.jdeprscan
Main - com.sun.tools.jdeps
Main - com.sun.tools.script.shell
Main - com.sun.tools.sjavac
Main - jdk.javadoc.internal.tool
Main - jdk.jfr.internal.tool
Main - jdk.tools.jaotc
Press 'Ctrl+Space' to show Template Proposals

```
public static void main(String[] args) {

}
```

Press 'Tab' from proposal table or click for focus

Imprimir mensagem no console

```
public static void main(String[] args) {

    System.out.println("Olá mundo");

}
```

Problems @ Javadoc Declaration Console

```
<terminated> OlaMundo [Java Application] C:\java\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_15.0.1.v20201027-0507\jre\bin\javaw
Olá mundo
```

Apresentações

- Apresentação:
  - Experiência profissional?
  - Em qual área pretende atuar? Tem alguma preferencia de tecnologia?
  - Tem alguma outra formação ou curso?
  - Linguagens em que programam? A quanto tempo?
  - Algo inusitado (Hobbies, Atividades extra curricular, costumes, etc....).

Como realizar a leitura do console

```
Scanner leitor = new Scanner(System.in);
```

Lendo dados digitados no console

Vamos criar uma classe que solicite o nome de um usuário e imprima uma mensagem de boas vindas.

Então vamos lá:

1) Crie uma nova classe chamada `LeitorConsole`.
2) Crie uma mensagem no console solicitando o nome do usuário;
3) Realize a leitura;
4) Imprima no console a mensagem: “Seja bem-vindo <nome_informado>”;

Resposta em 15 minutos

Resposta

```
public class LeitorConsole {

    public static void main(String[] args) {

        System.out.print("Informe seu nome: ");

        Scanner leitor = new Scanner(System.in);

        System.out.print("Olá " + leitor.next());
    }

}
```

Exercício

- Faça um programa que leia o nome, idade e data de nascimento do usuário (dd/mm/yyyy);
- O sistema deverá imprimir no console: “Nome do usuário: <nome>, tem <idade> de idade e nasceu no dia < data de nascimento>”;

Resposta em 15 minutos

E/S com JOptionPane

São pequenas janelas – diálogos – que interagem com o usuário.
Sua principal utilização se dá através dos 4 métodos estáticos – que não necessitam de uma instância para ser invocado – abaixo:

JOptionPane.showConfirmDialog( )	--	Faz uma pergunta ao usuário.
JOptionPane.showInputDialog( )	--	Requisita uma entrada.
JOptionPane.showMessageDialog( )	--	Mostra uma mensagem.
JOptionPane.showOptionDialog( )	--	Unificação dos 3 acima.

JOptionPane

showMessageDialog(Component parentComponent, Object message, String title, int messageType)

Component parentComponent – Componente sobre qual o dialogo será criado

Object message – A mensagem que será mostrada na tela. Normalmente é uma *String*, porém a mensagem será mostrada conforme seu tipo:

Component – O componente será mostrado conforme seu padrão.
Icon – Será mostrado a figura na tela como seu padrão.
Outros – O objeto será transformado em *String* conforme seu método *Object.toString();*

String title – O titulo do diálogo, será mostrado na barra de titulo.

JOptionPane

showMessageDialog(Component parentComponent, Object message, String title, int messageType)

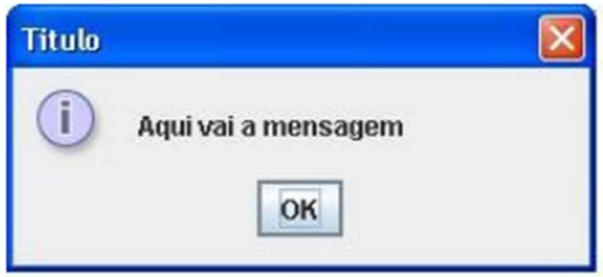
Int messageType – Define o tipo da mensagem – ícone do diálogo:

JOptionPane.ERROR_MESSAGE	–	0.
JOptionPane.INFORMATION_MESSAGE	–	1.
JOptionPane.WARNING_MESSGE	--	2.
JOptionPane.QUESTION_MESSAGE	–	3.
JOptionPane.PLAIN_MESSAGE	--	-1.

JOptionPane

Exemplo:

```
JOptionPane.showMessageDialog( null ,
                               "Aqui vai a mensagem" ,
                                "Titulo" ,
                                JOptionPane.INFORMATION_MESSAGE
                               );
```



JOptionPane

showConfirmDialog(Component parentComponent, Object message, String title, int optionType, int messageType)

Retorno: int – Valor inteiro do botão que o usuário clicou.

Component parentComponent – Componente sobre qual o dialogo será criado

Object message – A mensagem que será mostrada na tela. Normalmente é uma *String*, porém a mensagem será mostrada conforme seu tipo:

Component – O componente será mostrado conforme seu padrão.
Icon – Será mostrado a figura na tela como seu padrão.
Outros – O objeto será transformado em *String* conforme seu método *Object.toString();*

String title – O titulo do diálogo, será mostrado na barra de titulo.

JOptionPane

showConfirmDialog(Component parentComponent, Object message, String title, int optionType, int messageType)

Int optionType – Define os botões de resposta do diálogo, pode ser:

JOptionPane.YES_NO_OPTION – 0.

JOptionPane.YES_NO_CANCEL_OPTION – 1.

Int messageType – Define o tipo da mensagem – ícone do diálogo:

JOptionPane.ERROR_MESSAGE – 0.

JOptionPane.INFORMATION_MESSAGE – 1.

JOptionPane.WARNING_MESSGE -- 2.

JOptionPane.QUESTION_MESSAGE – 3.

JOptionPane.PLAIN_MESSAGE -- -1.

JOptionPane

showConfirmDialog(Component parentComponent, Object message, String title, int optionType, int messageType)

Int optionType – Define os botões de resposta do diálogo, pode ser:

JOptionPane.YES_NO_OPTION – 0.

JOptionPane.YES_NO_CANCEL_OPTION – 1.

Int messageType – Define o tipo da mensagem – ícone do diálogo:

JOptionPane.ERROR_MESSAGE – 0.

JOptionPane.INFORMATION_MESSAGE – 1.

JOptionPane.WARNING_MESSGE -- 2.

JOptionPane.QUESTION_MESSAGE – 3.

JOptionPane.PLAIN_MESSAGE -- -1.

JOptionPane

Exemplo:

```
JOptionPane.showConfirmDialog( null ,
                                "Mesagem" ,
                                "Titulo" ,
                                JOptionPane.YES_NO_OPTION,
                                JOptionPane.WARNING_MESSAGE
                               );
```

JOptionPane

showInputDialog(Component parentComponent, Object message, String title, int messageType, Icon icon, Object[ ] valoreSelecionáveis, Object valorInicial)

Retorno: String – String que o usuário digitou, ou o objeto selecionado.

Component parentComponent – Componente sobre qual o dialogo será criado

Object message – A mensagem que será mostrada na tela. Normalmente é uma *String,* porém a mensagem será mostrada conforme seu tipo:

Component – O componente será mostrado conforme seu padrão.
Icon – Será mostrado a figura na tela como seu padrão.
Outros – O objeto será transformado em *String* conforme seu método *Object.toString();*

String title – O titulo do diálogo, será mostrado na barra de titulo.

JOptionPane

showInputDialog(Component parentComponent, Object message, String title, int messageType, Icon icon, Object[ ] valorSelecionáveis, Object valorInicial)

Int messageType – Define o tipo da mensagem – ícone do diálogo:

JOptionPane.ERROR_MESSAGE	–	0.	
JOptionPane.INFORMATION_MESSAGE	–	1.	
JOptionPane.WARNING_MESSGE	--	2.	
JOptionPane.QUESTION_MESSAGE	–	3.	
JOptionPane.PLAIN_MESSAGE	--	-1.	

Icon icon – Ícone mostrado no diálogo.

JOptionPane

showInputDialog(Component parentComponent, Object message, String title, int messageType, Icon icon, Object[ ] valorSelecionáveis, Object valorInicial)

Object[ ] valorSelecionáveis – Vetor de objetos que será dispostos – em um *JComboBox* – para a seleção do usuário. Se *null, ao* invés do *JComboBox* o usuário poderá digitar sua entrada em uma caixa de texto.

Object valorInicial – Valor previamente selecionado, ou valor já digitado na caixa, se for o caso.

JOptionPane

Exemplo:

```
Object[] opcoes = {"um", "dois","tres","quatro"};

JOptionPane.showInputDialog( null , "Escolha" , "Titulo" ,
JOptionPane.QUESTION_MESSAGE , null , opcoes , "tres" );

JOptionPane.showInputDialog( null , "Escolha" , "Titulo" ,
JOptionPane.QUESTION_MESSAGE , null , null , "tres" );
```

JOptionPane

showOptionDialog(Component parentComponent, Object message, String title, int optionType, int messageType, Icon icon, Object[ ] options, Object initialValue)

Retorno: int – Valor inteiro do botão que o usuário clicou.

Component parentComponent – Componente sobre qual o dialogo será criado

Object message – A mensagem que será mostrada na tela. Normalmente é uma *String,* porém a mensagem será mostrada conforme seu tipo:

Component – O componente será mostrado conforme seu padrão.
Icon – Será mostrado a figura na tela como seu padrão.
Outros – O objeto será transformado em *String* conforme seu método *Object.toString();*

String title – O titulo do diálogo, será mostrado na barra de titulo.

JOptionPane

showOptionDialog(Component parentComponent, Object message, String title, int optionType, int messageType, Icon icon, Object[ ] options, Object initialValue)

Int messageType – Define o tipo da mensagem – ícone do diálogo:

JOptionPane.ERROR_MESSAGE	–	0.
JOptionPane.INFORMATION_MESSAGE	–	1.
JOptionPane.WARNING_MESSGE	--	2.
JOptionPane.QUESTION_MESSAGE	–	3.
JOptionPane.PLAIN_MESSAGE	--	-1.

Icon icon – Ícone mostrado no diálogo.

JOptionPane

showOptionDialog(Component parentComponent, Object message, String title, int optionType, int messageType, Icon icon, Object[ ] options, Object initialValue)

Object[ ] options – Vetor com os botões do diálogo.

Object initialValue – Botão previamente selecionado.

JOptionPane

Exemplo:

```
Object[] opcoes = {"um", "dois","tres","quatro"};

JOptionPane.showOptionDialog( null , "Escolha" , "Titulo" ,
JOptionPane.YES_NO_CANCEL_OPTION , JOptionPane.WARNING_MESSAGE ,null ,
opcoes , "tres");
```

Exercício

- Faça um programa utilizando JOP, que leia nome, idade e data de nascimento do usuário (dd/mm/yyyy);
- O sistema deverá imprimir no console: “Nome do usuário: <nome>, tem <idade> de idade e nasceu no dia < data de nascimento>”;

Resposta em 15 minutos

Perguntas?

Bibliografia básica

ECKEL, Bruce. Thinking in Java. New Jersey: Prentice Hall, 1998.
SIERRA, Kathy; BATES, Bert. Use a cabeça! Java. 2. ed. Rio de Janeiro: Alta Books, 2009-2010.
SINTES, Anthony. Aprenda programação orientada a objetos em 21 dias. 1. ed. São Paulo: Pearson Makron Books, 2002-2010.

Bibliografia complementar

HORSTMANN, Cay S.; CORNELL, Gary. Core JAVA. 8. ed. São Paulo: Pearson Prentice Hall, 2009.
DEITEL, Harvey M.,; DEITEL, Paul J. Java: como programar. 6. ed. atual. São Paulo: Prentice Hall, 2005.
POO, Danny C. C.,; KIONG, Derek Beng Kee,. Object-oriented programming and Java. Singapore: Springer, 1999-2001.
SIERRA, Kathy; BATES, Bert. Certificação SUN para programador Java 6: guia de estudo : (Exame 310-065).